

Designing and programming a package installer and dependency resolver tailored to Linux From Scratch systems and users

Shayan Nezami

February 1, 2026

Contents

1	Introduction	3
1.1	Background	3
1.1.1	Linux	3
1.1.2	Package managers	3
1.1.3	Linux From Scratch	4
1.2	Motivation	4
2	Design	5
2.1	Non-technical aims	5
2.2	Package maintenance	6
2.2.1	Modifying destdir	6
2.2.2	Using unionfs	7
2.2.3	Functional package managers	7
2.2.4	Conclusion	7
2.3	Dependency resolution	7
2.3.1	Maintaining a curated repository	8
2.3.2	Holding a package database	8
2.3.3	Using placeholders	8
2.3.4	Conclusion	9
3	Programming the package installer	9
3.1	Basic installation	9
3.2	Tracking file ownership	12

3.3	Uninstalling packages	13
4	Programming the dependency resolver	15
4.1	Storing metadata	15
4.1.1	conflicts.itch	16
4.1.2	depends.itch	16
4.1.3	provides.itch	16
4.2	Defining the SAT problem	16
4.2.1	The format of DIMACS CNF	17
4.3	Generating a CNF file	17
4.4	Full dependency resolution	19
5	The final product	20
5.1	Configuration files	20
5.2	Typical directory tree	20
5.3	Installation	22
5.3.1	The package installer	22
5.3.2	The dependency resolver	22
5.4	Example usage	23
6	Conclusion	28
6.1	Extent of success	28
6.2	Scope for improvement	29
6.3	Comparison with other package managers	30
A	Package installer	32
A.1	itch-installer	32
A.2	updatedb.py	34
A.3	uninstalldb.py	35
B	Dependency resolver	36
B.1	itch-resolver	36
B.2	make-cnf.py	36
B.3	get-positives.py	42

1 Introduction

1.1 Background

In this section, I will set out some key terms and definitions, and explain some crucial concepts, knowledge of which form prerequisites to the rest of the write-up. This is intended mostly towards readers with little knowledge about modern Linux systems.

1.1.1 Linux

Linux, while sometimes referred to as an operating system, like macOS and Windows, is in fact just a kernel. A **kernel** is, in simple terms, the core of an operating system, dealing with the CPU, and allocating memory to processes, among other essential operations. However, on its own, it does not provide a functional system, and is certainly not an operating system itself [1].

An operating system is a collection of software that provides a usable system to an end user. Therefore, the Linux kernel is often distributed along with large amounts of other software, in the form of a **Linux distribution**. There are hundreds, if not thousands, of such distributions, examples including Ubuntu, Debian, and Arch Linux. When I refer to a **Linux system**, this is not in reference to the Linux kernel, but rather a full operating system built on top of the kernel.

1.1.2 Package managers

All computer systems need software to function, so there must be a mechanism for users to install software. At its core, **software** is simply code, that is **compiled** ("translated") to a binary format that can be executed by the processor. Installing software, in its most basic form, just consists of copying those binary files to a location where the system recognises them as executable programs.

For Windows systems, most software comes with a dedicated installer (and uninstaller), which carries out this process for the application. However, this is very rarely the case on Linux systems. Software is often released as just source code, and the user is responsible for the installation.

Manual installation is not particularly difficult (the code simply needs to be downloaded, compiled, tested, then copied into place), however this can get tedious when repeated for hundreds of programs. Furthermore, this makes uninstallation very difficult, as there is no way to know which files were installed by which program, and dependency management becomes near impossible.

Dependencies are software or libraries that are required for a program to be installed. There are also **conflicts** between software, where two programs cannot be simultaneously installed on a system. Often, these occur between versions of packages.

For example, where p denotes a package, p_a may require p_c with version ≥ 3.0 , and p_b may conflict with p_c versions ≥ 3.5 . This means that for p_a, p_b, p_c to be simultaneously installed, p_c must have version v , where $3.0 \leq v < 3.5$.

Therefore, one of the defining features of a Linux distribution is the package manager they include. **Package managers** can perform all the tasks above, and more. Very often they have access to a repository of trusted and popular software, so the user can install, uninstall, and update software with just one command. For example, the following command in Arch Linux installs the web browser, Firefox:

```
# pacman -S firefox
```

Here, pacman - the package manager for Arch Linux [2] - gets the *package* for Firefox from its public repository, and then copies the required files into place, keeping records of what files it installed and where. This incredible ease of use is a massive draw towards Linux, and specifically existing Linux distributions.

1.1.3 Linux From Scratch

Linux From Scratch (LFS) is a project that provides an online, continuously updated, book with instructions on how to build a Linux system yourself, compiling and installing all software manually from source code [3], and not using any of the provided distributions - the vast majority of which come with some form of installer.

1.2 Motivation

Having recently installed a Linux From Scratch system, there is a clear gap for a package manager, which is not provided in the book. This is intentional, to bring the user's attention to compiling and installing the base software from scratch [4]. However, this is not sustainable if the system is to become usable long-term.

An advanced, practically usable system will at some point need complex software packages, including but certainly not limited to graphical toolchains, programming frameworks, and desktop environments. These come with many dependencies, shared libraries, and version conflicts.

Even without such large packages, as a system grows, so will the number of packages (even smaller ones) it has, which feed into the same issues. There are also problems associated with upgrading software, where all files relating to the old version need to be cleanly uninstalled, and replaced with the new version, without breaking the dependencies of any other currently installed software [4]. It is not sustainable to continue without a well-defined package manager at this point.

This drives users into three main groups:

1. They create a basic, personal package manager to suit their needs.
2. They install another distribution's package manager, such as Gentoo's Portage, or Arch's pacman [5].
3. They leave their LFS system as-is after installation, never using it for any practical purpose, rather leaving it as a completed educational experience.

All three of these come with significant, non-trivial downsides.

In the first case, these package managers tend to be simple and limited in scope, and while their creation does provide a good educational experience, they are often not capable of fully supporting a growing system.

Plus, programming ability is not a prerequisite to building an LFS system, so, while a minority, there are users who are capable of installing such a system, but not creating a package manager for it. While there are no formal studies on the proportion of Linux users who are not programmers, online discussions seem to suggest the existence of a notable minority [6].

The second case has the benefit of providing a highly reliable, popular, and documented package manager, often with all the user's needed features and more. However, there are two key downsides here.

Firstly, this in effect turns the user's LFS system into a version of the distribution providing the package manager, eliminates many of the purposes of LFS, which is about creating and maintaining your own

system, with full control over all software installed. The sheer complexity of these package managers removes such control from the users, and while it does often produce a working system, the user has in effect just followed a very long path to installing that distribution.

Secondly, and perhaps most crucially, these package managers have been specifically designed for their own distribution, and an LFS system is by definition different. These systems may handle certain cases differently, or have different directory structures, among other subtle differences. Since the package manager has not been made for, and is not commonly used with, LFS systems, there tends to be a lack of support, with a very small user base that can help with any issues that will eventually arise.

For these reasons, I came to the decision to write a package manager, dedicated to Linux From Scratch systems, addressing all the shortfalls identified.

2 Design

2.1 Non-technical aims

At the very start of the project, I established that the most basic level of success for the package manager can be determined by whether it can install and remove packages. This is trivial to test - I just need to install a sample of packages, verify that they work correctly, then remove them, and verify that they are no longer present on the system.

However, it is not particularly challenging to create such a package manager, and many already exist. Therefore, the following aims will act as my criteria for establishing a more sophisticated level of success for the project. Almost all of them stem from Aim 1, the need to target Linux From Scratch users and systems, since the problem I am aiming to solve with this project is the lack of a suitable package management solution for Linux From Scratch users, as established in Section 1.2.

These criteria will also be used to guide the development process, ensuring the artefact created fits my goals as closely as possible.

1. It must be targeted towards LFS systems. While it should ideally be usable on most Linux systems, decisions should be guided by what is needed on an LFS system, and what the LFS user base would prefer.
2. The compilation ("building") of software should be left to the user, meaning precompiled binaries should never be distributed. Linux From Scratch is based on the principles of compiling ones own software, and this is a key requirement if a package manager is to be used by LFS users.
3. Users should be able to patch and modify source code as they desire prior to installation, with the package manager placing no restriction on this.
4. The compiled software should be stored as *packages* of some form for future use, so that the user does not have to rebuild software to reinstall it. Note that this does not defy Aim 2, since these packages will have been compiled by the user themselves, configured as they desire, rather than externally compiled, then distributed.
5. There should be no central server the package manager needs to interact with to function. It should be able to run wholly locally, with no limitations.
6. It must provide a method for full and clean uninstallation, removing all traces of the software, while not affecting any other programs.

7. Dependencies and conflicts must be tracked and managed appropriately, as a key purpose of this project.
8. It must not simply be a derivative of an existing package manager - there must be something unique about it, or a non-trivial reason to use it instead of another existing package manager. This has been partially addressed in Section 1.2, but it should be continuously thought about.

Aims 1 and 8 need to be tested qualitatively, with a personal judgement being used to establish whether they have been successful. This should be apparent when comparing the functionality of this package manager to other, existing ones, especially to do with the design decisions that have been made.

Aims 5 and 7 need to be tested by examining the source code I have produced, and ensuring the functionality conforms to their specifications.

The other aims can all be tested objectively, by attempting to use the functions they describe on the package manager for a sample of different packages. They will be classed as successful at the end of the project if those functions work for all packages tested.

These aims are not completely independent of each other, and should not be thought of as such. Instead, they build on and complement each other. For example, allowing users to compile packages allows them to patch their source code as well. Aim 8, the need to be original, in some ways derives from Aim 1, since aiming the package manager specifically towards Linux From Scratch users is something that has not been done before in a complete, distributable package manager. Further, most of the aims contribute towards meeting Aim 1, since they are part of what targets this package manager for LFS specifically.

I chose the name *Itch* for this package manager to reflect the importance of Aim 1, since it is being built for Linux From *Scratch*.

2.2 Package maintenance

Through my research of existing package managers, I have identified three promising methods employed by package managers to install and subsequently uninstall packages. These generally all provide some method to track what files are installed, and where, so that they can be removed on uninstallation.

These are generally independent of the method for dependency resolution, which will be discussed in Section 2.3.

2.2.1 Modifying `destdir`

This method is used by many package managers made for commercial Linux distributions, such as `pacman` [2]. This is a very simple method, consisting of configuring the package to be installed in the correct, final, location, but modifying the `DESTDIR` environment variable upon installation, so that the package is instead placed in a "fake" location where it will not function properly [7].

From there, an archiving utility, often `tar`, is used to create an archive of the files in this fake directory. This archive is then unpacked in the correct installation location, installing the software as normal. However, now the archive still exists, and within it, it holds a copy of every file that was installed, and where they were placed. This makes uninstallation very simple, as these files are simply iterated over and deleted [8].

2.2.2 Using unionfs

This is a unique approach to package management, and follows the same basic principles as the fake installation location method, but differs greatly in implementation [9].

The current file system is accessed read only, and a new, empty, file system is created, which can be written to. A union between these two is made using the unionfs utility. Therefore, when the package is installed to the union, the union will contain the new state of the system, with the new package installed, and the previously empty file system will contain all of the installed files.

These can then be stored in some format, providing all of the files installed by the package for easy uninstallation. Once the user is happy, the union file system can be copied to the main file system (that was previously accessed read only). This allows easy rollbacks in case of an error mid-install.

The main benefit this has over the DESTDIR method is that files are **forced** to be installed in the empty file system, while a poorly packaged program may ignore the DESTDIR environment variable. However, this is rare, and seems to be more of a theoretical advantage rather than a practical one.

2.2.3 Functional package managers

Functional package managers have been gaining popularity in recent years, and they are based on the principle that all configuration and instructions given to packages must be stored in one central file.

This file is then hashed, and that hash corresponds to the unique state of the system [10]. This is very useful for creating reproducible builds, and makes debugging much simpler since a configuration can be recreated on another device with ease.

While this is a fascinating concept, I made the decision not to research it much further, since these package managers do not fit within my fundamental aims for *Itch*, which is to complement a Linux From Scratch system, simplifying tasks. This would be introducing a whole new paradigm, and users would be much better served installing an existing, developed, functional package manager if they want a system like this, such as Nix.

2.2.4 Conclusion

Although the unionfs method was fascinating from a technical point of view, I decided to use the DESTDIR method over unionfs. The main reasons were that it is much better documented in case any problems arise, it is much simpler to understand and debug, particularly for users of the program who should ideally understand how it works in full, and the unionfs method did not provide any significant advantages over the DESTDIR method, despite the added complexity.

2.3 Dependency resolution

Now, I must consider the problem of dependency resolution. Each package can depend on, or conflict with, certain other packages, meaning certain packages must be installed with each other, while other packages cannot coexist on a system. To make this more complex, these are most often bounded by versions. See Section 1.1.2 for a deeper explanation.

This means that package managers must come with a method to *resolve* dependencies, meaning to find what packages - and in what versions - need to be installed on the system for a new package to be added.

Dependency resolution is an NP-complete problem [11], meaning in practical terms that it cannot reliably

be solved "quickly" (there is no known polynomial time solution). Therefore, shortcuts and optimisations are often taking to make it tractable.

2.3.1 Maintaining a curated repository

The vast majority of popular Linux distributions employ this method, where the maintainers of the distribution maintain an online repository, containing one version of each package available, ensuring that those versions all work together. This greatly simplifies the problem of dependency resolution, since package versions are no longer a concern.

Another reason this method is very commonly used in more mainstream distributions is that the package set becomes universal among systems of the same version, making problems easier to diagnose and debug.

Moreover, this fits very well with the philosophies of some distributions. For example, Arch Linux operates on a rolling-release model, always using the very newest software, so this curated repository only contains the newest usable builds [12]. On the opposite end of the spectrum, Debian focuses on stability, so a curated selection of tested, mature packages enables the creation of very robust systems [13].

However, this does not at all fit within the Linux From Scratch philosophy, which is about users creating their own system as they please. *Itch* is simply intended to be a tool to help users with this, and therefore it should not limit the range of software users can install, or curate package repositories based on release version. This choice should be fully in the hands of the user.

Furthermore, since this relies on an online repository of curated software, it violates Aim 5.

2.3.2 Holding a package database

Another method consists of holding a database with the required dependency information for a number of packages (available in many versions). This differs from the curated repository method, in which one version of each package is stored which will work with each other.

This allows the user to choose what packages they install, and with what version. Gentoo operates a similar system, using an *ebuild* repository, which contains *ebuild* files with the metadata of packages [14] - most of which have multiple available versions.

However, it does require than an online database to be maintained, violating Aim 5. While this can be mitigated, by allowing users to build their own database (keeping the database in a human-readable format), and by allowing the file to be downloaded once and repeatedly used, meaning the package manager does not need to interact with an online sever every time it runs, it still does violate the spirit of the aim, preventing the package manager being from functioning wholly locally.

2.3.3 Using placeholders

The problem with not holding any form of package database is that the system has no information about packages that could satisfy dependencies. Consider a system (using the notation $p_x - y$ to refer to package x , version y) where, the installed packages are $\{p_a - 3.0, p_b - 1.2\}$, and $p_c - 2.0$ is to be installed. However, $p_c - 2.0$ requires $p_a - v > 3.5$ and $p_d - v \leq 1.0$. Now, the system does not know about what packages exist within those ranges, and whether they depend on any other packages, or conflict with any existing packages. This makes dependency management very difficult.

In this case, placeholders can be used, meaning the resolver would simply output the version ranges that a satisfying package must fit into. However, this is a one-way relationship. Where the set S is a satisfying

set of packages, and S_p output of the program involving placeholders,

$$S \implies S_p, \quad S_p \not\Rightarrow S. \quad (1)$$

This is because the dependency information of those packages in the range is not known, nor is the actual package versions within the range.

This does not result in a useful package manager. For large packages with hundreds of (often nested) dependencies, it creates a very arduous path for the user.

Imagine a package $p_e - 4.2.1$ is to be installed, and the package manager identifies that it requires $p_e - 2.0.0 < v \leq 3.2.0$. Trying to install that, the package manager can only then inform the user that it requires p_f . However, the version of p_f the user chose to install ends up conflicting with $p_b - 2.0$, which is installed on the system. The system will have no method to detect these in advance, while a system with a package database would instantly output a satisfying arrangement, if one is available.

Very often, this method will be more time-consuming and frustrating than handling the dependencies yourself, or finding a satisfying arrangement online.

2.3.4 Conclusion

This is a tough situation, where either Aim 5 must be (at least partially) defied, or the dependency resolver produced will not be practically usable.

I can immediately remove the first option, of maintaining a curated repository, since that goes too far against the purpose of a Linux From Scratch package manager (Aim 1), and in effect creates a new distribution. The third option of using placeholders is also not reasonable, since it creates a borderline unusable program.

Therefore, I have no option but to hold a package database, containing just the dependency information for packages. This will be in a form that can be downloaded once, and extended by users. This database can start with data scraped from other package manager databases, and extended with user contributions.

However, since this does go against the concept I originally had for the package manager, I have decided it would be best to split the package *installer* (the installation and uninstallation components) and the dependency resolver into separate projects. They will still work with each other as a unified package management system, but they should also be capable of working independently, and in conjunction with other package management tools.

Besides the obvious benefit of the package installer component not relying on an external database, this also better follows the Unix philosophy, where each program should complete just one task well [15]. It is also more useful for users who only require one half of the system, and use a different method for either dependency resolution or package installation and removal.

3 Programming the package installer

3.1 Basic installation

The principles of building an installing software can be boiled down to the following steps: (patch), configure, build, (test), install. Since *Itch* is focused on ensuring users still compiling their own code (Aim 2), and allowing patching as desired (Aim 3), it will not interfere with any of these steps until *install*.

To make this clear to the user, I created procedures for *configure*, and *install*:

```

1  install_prefix="$HOME/prog/school/epq/dest/"
2
3  configure() {
4      echo "configure as normal, ensuring the following is added:"
5      echo "    --prefix=$install_prefix"
6  }
7
8  build() {
9      echo "make as normal"
10 }

```

This also reminds the user to specify the install prefix - the location the software will be once fully installed - to the configure command. It is currently set to a location within my home directory to allow for testing without risk of damaging my system, but this will be changed to /usr (the default prefix for LFS systems), with a configuration file allowing it to be changed, once the development is complete.

The code below performs the installation, with a method similar to that described in Section 2.2.1.

```

1  temp_dir="/tmp/itch/build"
2
3  install() {
4      rm -rf "$temp_dir" # in case not properly cleaned before
5      mkdir -p $temp_dir
6
7      make DESTDIR="$temp_dir" install # install in temp dir
8
9      findname
10
11     sudo mkdir -p "$pkgpath"
12
13     sudo tar czf "$itchpkgpath" -C "$temp_dir$install_prefix" .
14
15     installpkg
16 }
17
18 installpkg() { # existing tarball in pkgpath
19     cd $install_prefix
20
21     # finds existing files that will be overwritten
22     for file in $(tar tzf "$itchpkgpath" | grep -v '/$'); do
23         if [ -e "$file" ]; then
24             printf "Overwrites '%s'\n" "${file#./}"
25         fi
26     done
27
28     read -p "Proceed? (yes/NO): " confirmation
29
30     if [ "$confirmation" = "yes" ] || [ "$confirmation" = "y" ]; then
31         tar xf "$itchpkgpath"
32     else
33         echo "Aborted!"
34     fi
35
36     rm -rf "$temp_dir"
37 }

```

The following process is followed by the code above to install packages:

1. A temporary installation directory is created in `/tmp/itch/build`, and the package is installed there first. This directory is placed in `/tmp` as it is "made available for programs that require temporary files" [16].
2. The tar archive [17] (a gzipped tarball) of the files to be installed is stored in `/var/lib/itch/pkgs/{package name}/{package name}.itchpkg`. This both allows the package to be reinstalled without recompilation (Aim 4), and holds the data about files created during installation. It is located in `/var` as "`/var` contains variable data files" [16], and these files are updated while the program runs.
3. The files to be installed are compared with any files already installed, finding the files that will be overwritten, so the user can cancel the installation if that would be problematic. The for loop for this was heavily inspired by `qpkg`'s [8] `verify` function [18].
4. The tarball is unpacked in the final installation directory, completing the installation, and the temporary build directory is removed.

The following procedure determines the name of the package being installed, by either using the name of the current directory, or the parent directory if currently inside a build directory. Since this is not fully robust, the user has the chance to override its output.

```
1  findname() {
2      pkgname=$(basename "$(pwd)")
3
4      # tries to find package name (either current dir or parent if in build dir)
5      if [ "$pkgname" = "build" ]; then
6          pkgname=$(basename "$(dirname "$(pwd)")")
7      fi
8
9      echo "Enter package name (blank for [$pkgname]):"
10     read input
11
12     if [ "$input" != "" ]; then
13         pkgname="$input"
14     fi
15 }
```

A switch statement calls the required procedure based on the flags the user passes into the program.

```
1  case "$1" in
2      "-c")
3          configure ;;
4      "-b")
5          build ;;
6      "-i")
7          if [ -z "$2" ]; then
8              install
9          else
10             getname "$@"
11             installpkg
12          fi ;;
13      *)
14          echo "Error!"
15      ;;
```

The `installpkg` procedure is called directly if the name of a package that has already been compiled by the user is specified, skipping the building stage. In this case, the `getname` procedure is called, setting variable based on the name of the package provided, and ensuring it is valid.

```

1  getname() {
2      pkgname="$2"
3
4      pkgpath="/var/lib/itch/pkgs/$pkgname"
5      itchpkgpath="$pkgpath/$pkgname.itchpkg"
6
7      if [ ! -f "$itchpkgpath" ]; then
8          echo "Error!"
9          exit
10     fi
11 }

```

3.2 Tracking file ownership

Currently, the `uninstall` method will just consist of deleting all the files that were installed by a package. This brings about a problem: some files are provided by multiple packages. These should not be deleted upon the removal of one package that provided them if another package that also provided them is still installed.

There is also a second, though less significant, problem in the removal of directories. These should only be removed if they will be empty once all files to be deleted are removed, to prevent another program's files residing in the same directory being deleted. The approach `qpkg` takes is simply not deleting directories [8], and while this still works, it is not very elegant.

Both problems can be solved by tracking which packages provide each file.

I chose to use `gdbm` [19] for the database holding this information. It provides a quick, lightweight, NoSQL method to retrieve data, and is already installed in a default LFS system [20].

The `pkgfiles` array is populated inside `install`, and this `updateshared` procedure is called just prior to final installation in the final directory.

```

1  updateshared() {
2      sharedir="/var/lib/itch/shared"
3      dbdir="$sharedir/files.gdbm"
4
5      # create dir and file if not already existing
6      sudo mkdir -p "$sharedir"
7      sudo touch "$dbdir"
8
9      for file in ${pkgfiles[@]}; do
10         gdbmout=$(echo "fetch $file" | sudo gdbmtool "$dbdir" 2>/dev/null)
11
12         echo "store $file \"$gdbmout $pkgname\" | sudo gdbmtool "$dbdir"
13     done
14 }

```

This uses `gdbmtool` (a command line interface to `gdbm`) to add the name of the installed package onto

the list of packages that provide a certain file. This table maps each installed file (the key) to this list (the value).

The database is located at `/var/lib/itch/shared/files.gdbm`. It is in a "shared" directory to show that this file is used by all packages.

However, upon testing, it would take minutes for this procedure to run for larger packages. This was since every time `gdbmtool` is called (twice per file), the database is opened and closed, and while the actual reading and writing is very quick, this overhead greatly slowed down the process.

Therefore, I replaced this procedure with a short python script - `updatedb.py` - performing the same task.

```
1  import sys
2  import dbm.gnu
3
4  dbdir = sys.argv[1]
5  pkgname = sys.argv[2]
6
7  db = dbm.gnu.open(dbdir, 'c');
8
9  for i in range(3, len(sys.argv)):
10     file = sys.argv[i]
11
12     gdbmout = db.get(file)
13
14     if gdbmout is not None:
15         pkgs = gdbmout.decode().split(' ')
16     else:
17         pkgs = []
18
19     # don't add pkgname if already existing - eg for reinstallations
20     if pkgname not in pkgs:
21         pkgs.append(pkgname)
22
23     db[file] = " ".join(pkgs)
24
25  db.close()
```

Python - unlike bash, which is only used for scripting - provides a well-documented interface to `gdbm` [21] such that the database can be opened once at the start of the program, and only closed at the end. This greatly reduced the time taken to update the database to milliseconds, from minutes.

This script is executed from inside the main bash file as follows:

```
1  printf "%s" "${pkgfiles[@]}"
2  | sudo python "$pythonupdatedir" "$filedbdir" "$pkgname"
```

The location of the database and package name are passed to it as command line arguments, while the package files are piped into it, as otherwise the number of files would often exceed the maximum length of command line arguments.

3.3 Uninstalling packages

The only reason such lengths have been taken to install packages, rather than simply issuing the installation command

```
1 # sudo make install
```

is to ensure that these packages can be cleanly uninstalled, leaving no traces behind that could cause problems for other packages, or newer versions of the same package.

Before attempting to uninstall a package, it must first be verified that it is installed in the first place. The contents of `/var/lib/itch/pkgs/` cannot be used for this, as they simply contain all archives that have been created. One may have been created but not installed, or installed but later uninstalled. The `.itchpkg` archives persist here intentionally, to allow easier installation again, but this does mean another method for tracking installed *packages* (not files) is needed.

I decided to also use `gdbm` for this, linking each package to its installed version.

```
1 echo "store $pkgname_name $pkgname_version"
2 | sudo gdbmtool "$pkgdbdir" 2>/dev/null
```

This does not require a separate python script, as the database is only accessed once per package installed.

The database is stored in `/var/lib/itch/shared/pkgs.gdbm`, again because this is shared between all packages.

```
1 sharedir="/var/lib/itch/shared"
2 pkgdbdir="$sharedir/pkgs.gdbm"
```

Arch Linux's `pacman` stores this data in the form of a tarball containing each package as a directory [2], however I chose to use this `gdbm` database instead since the hashing provided, and not needing to unpack the archive every time, makes it quicker to check whether a package is installed.

Having built this database, I then wrote the uninstallation procedure. A python script is used here as it allows the database to be opened just one time, have the operations performed, and then be closed, rather than opening and closing the database each time as would happen had I just used `bash`. This translates into a very significant (around tenfold in my testing) time saving.

```
1 uninstall() {
2     gdbmout="$(echo "fetch $pkgname" | sudo gdbmtool "$pkgdbdir" 2>/dev/null)"
3
4     if [ "$gdbmout" = "" ]; then
5         echo "Error! Package not installed to begin with!"
6         exit
7     fi
8
9     pkgfiles="$(tar tzf "$itchpkgpath")"
10
11     toremove="$(printf "%s" "${pkgfiles[@]}")"
12     | sudo python
13     "$pythonuninstalldir" "$filedbdir" "$pkgname" "$itchpkgpath")"
14
15     echo "$toremove"
16
17     IFS=$'\n' read -rd '' -a toremovearr <<< "$toremove"
18
19     for file in "${toremovearr[@]}; do
20         echo "$file"
21         sudo rm -rf "$installprefix/$file"
22     done
```

```

23     pkgname_name="${pkgname%%-[0-9]*}"
24     echo "delete $pkgname_name" | sudo gdbmtool "$pkgdbdir" 2>/dev/null
25 }
26

```

After confirming that the package is installed, the package's files are taken from the .itchpkg archive made in /var/lib/itch/pkgs/{package name} during installation. At this point, the python script uninstalldb.py is executed.

```

1  import sys
2  import dbm.gnu
3
4  dbdir = sys.argv[1]
5  pkgname = sys.argv[2]
6  itchpkgpath = sys.argv[3]
7
8  pkgfiles = [line.strip() for line in sys.stdin]
9
10 db = dbm.gnu.open(dbdir, 'c');
11
12 for file in pkgfiles:
13     gdbmout = db.get(file)
14
15     if gdbmout is not None:
16         pkgs = gdbmout.decode().split(' ')
17     else: # must have been an error
18         pkgs = []
19         continue
20
21     pkgs.remove(pkgname)
22
23     if len(pkgs) == 0:
24         print(file)
25         del db[file]
26     else:
27         db[file] = " ".join(pkgs)
28
29 db.close()

```

This opens the files.gdbm database, and removes all occurrences of package being uninstalled. It also prints any files that were only provided by this package, so they can be captured by the main bash procedure, and deletes those entries from the database.

These files that are returned are then all deleted, and the package is removed from the pkgs.gdbm database, as it is no longer installed.

4 Programming the dependency resolver

4.1 Storing metadata

As previously established, there needs to be a database with multiple versions of each available package, storing dependency information. Here, I will specify the format in which this is stored.

Firstly, there will be a gdbm database linking each package name with its available versions (`/var/lib/itch-resolver/global.gdbm`).

The data for each of these package and version is stored in the general directory: `/var/lib/itch-resolver/pkgs/{package name}/{package version}`. This directory will contain three files: `conflicts.itch`, `depends.itch`, and `provides.itch`.

Together, these three files contain all the required information to resolve dependencies and output a combination of packages that all will work together. This format is easy to parse in a program, and as compact as possible, minimising the storage size needed while holding all of the necessary information.

4.1.1 conflicts.itch

Each line of this file is in the format:

```
1 {conflicting package name} {lowest conflicting version} {highest conflicting version}
```

The lowest version can be 0, and highest can be left out if not applicable.

These are the packages and versions that cannot be installed together with this package.

4.1.2 depends.itch

This has a very similar format. Each package that is required for the target package has a line in the format:

```
1 {required package name} {lowest version required} {highest acceptable version}
```

4.1.3 provides.itch

This stores programs that are provided by a certain package, and may be required by other packages. This file has the even simpler format:

```
1 {provided program name} {provided program version}
```

No range is required as each package version provides can only provide another package in one version.

4.2 Defining the SAT problem

A dependency resolution problem is expressed as a set of constraints: one package requires another to run, two package versions cannot co-exist, a certain package must be newer than a given version to support a specific version of another package.

These can be encoded as a boolean satisfiability (SAT) problem. This is clear, since both the SAT problem, and dependency resolution, are NP-complete, meaning they reduce to each other in at most polynomial time [11]. This is useful, since there exists programs called SAT solvers, which are highly optimised to compute large SAT problems.

Therefore, by encoding all the dependency constraints into conjunctive normal form (CNF - the format accepted by SAT solvers), the problem can be given to a SAT solver to find a satisfying arrangement of packages.

I will pass this generated CNF to MiniSAT [22], a popular, powerful but lightweight SAT solver with a well-documented shell interface [23].

The following rules will be encoded [24]:

1. A package's dependencies must be met, or the package not installed.
2. None of a package's conflicts should be installed, or the package not installed.
3. Only one version of any package can be installed.
4. Each package already installed on the system must remain installed in some version (this does not have to be the current one).
5. The package being installed should be installed with the given version.

4.2.1 The format of DIMACS CNF

CNF, or conjunctive normal form, is a format for expressing a boolean expression, where variables (which can only take true/false values) are expressed in a *conjunction* of *disjunctions*. For a disjunction to evaluate to true, at least one variable inside must be true. For a conjunction to do the same, *every* variable inside it must be true. Practically, this means CNF contains multiple lists of variables, in each of which at least one must be true in a satisfiable state. Variables can also be negated: this means that a false variable maps to true, and vice versa [25].

MiniSAT takes input of a CNF file in DIMACS format. DIMACS is a file format for representing CNF, where each line contains a list of integers (which correspond to the boolean variables in disjunction), with the lines in the file are in conjunction together. A negative integer encodes the negation of a variable. All lines must also end with an 0, but this is not a problem variable.

4.3 Generating a CNF file

First, we must create a mapping between each package-version and an integer, which will represent the package in the DIMACS file.

```
1 count = 1
2 for pkg in pkgs:
3     versions = global_db[pkg.decode()].decode().split()
4
5     for version in versions:
6         pkg_to_num[pkg.decode(), version] = str(count)
7         num_to_pkg[count] = (pkg.decode(), version)
8
9         count += 1
10
11     set_provides(pkg.decode(), version)
```

The provides data for the packages were also read and stored in this loop, so that they are finished before the conflicts are encoded (if p_a conflicts with p_b , and p_c provides p_b , then p_a also conflicts with p_c , but there is no way of knowing this before the provides data has been encoded).

At this point, we can also encode the dependency and conflict information for the packages.

```

1 for pkg in pkgs: # needs second loop so mappings (and provides) are complete!
2     versions = global_db[pkg.decode()].decode().split()
3
4     for version in versions:
5         encode_deps(pkg.decode(), version)
6         encode_conflicts(pkg.decode(), version)
7
8     # encode "only one version of each package"
9     for i in range(0, len(versions)):
10         for j in range(i+1, len(versions)):
11             sat_clauses.append("-" + pkg_to_num[pkg.decode(), versions[i]] + " "
12                               + "-" + pkg_to_num[pkg.decode(), versions[j]] + " 0")

```

Then, the required clauses for local packages are installed, ensuring the packages that are installed remain installed in some version.

```

1 local_pkgs_db = dbm.gnu.open(local_pkgs_db_dir, 'c')
2 local_pkgs = local_pkgs_db.keys()
3
4 full_closure = set()
5
6 for pkg in local_pkgs:
7     # encode "each package installed should be installed (but whatever version)"
8     versions = global_db[pkg.decode()].decode().split() # all available versions
9
10    new_clause = ""
11    for version in versions:
12        new_clause += pkg_to_num[pkg.decode(), version] + " "
13
14        full_closure.update(get_closure(pkg.decode(), version)) # get closure of each
15        # installed/installable pkg-ver
16    new_clause += "0"
17
18    sat_clauses.append(new_clause)
19
20 full_closure.update(get_closure(new_pkg_name, new_pkg_version))

```

Above, these packages are also added to a set representing a closure. This closure is expanded by the `get_closure` method to contain every dependency of the local packages, and all dependencies of those dependencies, and the dependencies of those, etc. This is also done for the package being installed.

With these, the program adds negated clauses for all packages that do not appear in the closure, ensuring they are not installed. This fixes an issue that came up during testing, where the SAT solver would arbitrarily define unnecessary packages as true. While this would result in a valid system state, the user should only be prompted to install the dependencies they need, rather than all of the packages that could coexist on their system.

```

1 for pkg_num, (pkg, version) in num_to_pkg.items():
2     if (pkg, version) not in full_closure:
3         sat_clauses.append("-" + str(pkg_num) + " 0")

```

The program terminates by printing all of the SAT clauses to standard output, and storing the mappings of DIMACS integers to packages in the closure in a json file.

```

1 for clause in sat_clauses:

```

```

2     print(clause)
3
4 num_to_pkg_closure = dict()
5
6 for (pkg, ver) in full_closure:
7     num_to_pkg_closure[pkg_to_num[pkg, ver]] = (pkg, ver)
8
9 with open(sys.argv[5], 'w') as file:
10    json.dump(num_to_pkg_closure, file)

```

4.4 Full dependency resolution

The CNF-generating program defined above needs to be integrated with a MiniSAT to perform the actual resolutions and output a list of package-versions to be installed.

```

1 mkdir -p "$tempdir"
2
3 sudo python "$installdir/make-cnf.py" "$pkgmdir/" "$itchdir/" "$1" "$2"
4     "$tempdir/mappings.json" > "$tempdir/output" # inputs are pkg and version
5 minisat "$tempdir/output" "$tempdir/minisat_out" > /dev/null # don't print sat solver output
6 (grep -oE '(\^| ) [0-9]+' "$tempdir/minisat_out" | tr -d ' ') > "$tempdir/positives"
7
8 python "$installdir/get-positives.py" "$tempdir/mappings.json" "$tempdir/positives"
9
10 rm -rf "$tempdir"

```

This bash script redirects the output of the make-cnf script into a temporary output file, passes that file to MiniSAT, which generates a list of all variables (representing packages) and whether they are true or false (representing to be installed or not). This file is trimmed to only include the positive, true, variables (packages to be installed), and this is passed to another python script that uses this list, as well as the json file mapping variables to package-versions, to print the complete list of packages that should be installed on the system to provide a minimal, unproblematic state. That script is below:

```

1 import json
2 import sys
3
4 print("Suggested system state [name, version]:\n")
5
6 with open(sys.argv[1], 'r') as file:
7     num_to_pkg = json.load(file)
8
9 with open(sys.argv[2], 'r') as file:
10    for line in file:
11        line = line.rstrip() # remove trailing \n
12
13        if line == "0":
14            break
15
16    print(num_to_pkg[line])

```

Note that all temporary files are placed in a specific directory, which is deleted upon the program's termination, so that unnecessary space is not taken up on the user's system.

5 The final product

5.1 Configuration files

During development, I hardcoded the file paths of the resources that need to be accessed by the various programs. However, this is not suitable for distribution, as users should be able to modify these variables in one place and have it take effect across all program files, and it is best practice to ensure users do not need to modify source code files.

Therefore, I created two configuration files, one for the installer, and the other for the resolver, holding these values. A typical setup (which is being used on my system) is shown below.

```
1 installprefix="/usr"
2 tmpdir="/tmp/itch/build"
3
4 pkgpath="/var/lib/itch/pkgs"
5 sharedir="/var/lib/itch/shared"
6
7 itchinstallerdir="/usr/lib/itch-installer"
```

```
1 installdir="/usr/lib/itch-resolver"
2
3 pkgmdir="/var/lib/itch-resolver"
4
5 itchdir="/var/lib/itch"
6
7 tmpdir="/tmp/itch-resolver"
```

These values are loaded by the relevant bash scripts as follows, based on the configuration file needed:

```
1 source "/etc/itch/itch.conf"
```

```
1 source "/etc/itch/itch-resolver.conf"
```

Note that the installprefix is where packages installed by the installer will be located (/usr is the default for Linux From Scratch systems).

Also note that the dependency resolver's configuration file contains an entry for itchdir, the directory where the files of the package installer are contained. This is not required, and if that directory is empty the dependency resolver will simply assume that no packages are currently installed on the system. Otherwise, if it is provided, it allows the dependency resolver to ensure already installed packages do not conflict with the one being installer, and vice-versa.

5.2 Typical directory tree

This diagram shows where all of the files associated with both the installer and dependency resolver are to be installed on a typical system. The configuration files allow scope for changing some of these, however this layout follows the Linux Filesystem Hierarchy Standard [16] and is therefore recommended.

This clearly shows the separation between the installer and resolver components, since they rely on and use separate directories.

The package archives, storing installation information for future removal, and providing precompiled archives for reinstallation, are found under /var/lib/itch/pkgs/, and /var/lib/itch/shared/ contains

```

/var/lib/
├─ itch/
│  └─ pkgs/
│     └─ [package name]-[package version]/
│        └─ [package name].itchpkg
│  └─ shared/
│     └─ files.gdbm
│     └─ pkgs.gdbm
└─ itch-resolver/
   └─ pkgs/
      └─ [package name]/
         └─ [package version]/
            ├── depends.itch
            ├── conflicts.itch
            └─ provides.itch
   └─ global.gdbm

/tmp/
├─ itch/
│  └─ build/
└─ itch-resolver/

/usr/
├─ bin/
│  ├── itch-installer
│  └─ itch-resolver
└─ lib/
   ├── itch-installer/
   │  ├── uninstalldb.py
   │  └─ updatedb.py
   ├── itch-resolver/
   │  ├── get-positives.py
   │  └─ make-cnf.py
└─ etc/
   ├── itch/
   │  ├── itch.conf
   │  └─ itch-resolver.conf

```

Figure 1: The directory tree for *Itch*

information of the packages currently installed on the system, and the files that are installed (and which package(s) they belong to).

The dependency information is held at `/var/lib/itch-resolver/`. The `depends`, `conflicts`, and `provides` files are stored in the `pkgs/` subdirectory, and the `global.gdbm` database holds information about what packages, and in what versions, the system has information about.

The temporary directory `/tmp/itch/build/` is used as the fakeroot location for redirecting the installation of a package, and `/tmp/itch-resolver/` stores the temporary files created while the dependency resolver runs.

The `/usr/bin/` directory contains the executable shell scripts for both programs, and `/usr/lib/` contains the python scripts they call during their runtime.

Finally, `/etc/itch/` contains the configuration files for both programs. They are not in the users `.config` directory, since *itch* is designed to work system-wide.

5.3 Installation

Installation is generally simple, but has to be done separately for the package installer and dependency resolver components. This is by design, so that users can install only one, but not the other, if desired.

As dependencies, both components require Bash, Python 3, and GDBM. Additionally, the dependency resolver requires MiniSAT. MiniSAT is the only dependency that is not installed in a default Linux From Scratch installation.

This software has only been tested on a Linux From Scratch and an Arch Linux system. There is no reason that it should not also work on other distributions, but it is only intended to be used on Linux From Scratch, and so some subtleties of other systems may unexpectedly interact with the programs' functionality.

5.3.1 The package installer

The package installer will come with three files: `itch-installer`, `uninstalldb.py`, and `updatedb.py`.

`itch-installer` should be copied into `/usr/bin/`, and the two python scripts into `/usr/lib/itch-installer/`. This directory should be made if not already present.

The configuration file should be created in `/etc/itch/itch.conf`, containing the fields specified in Section 5.1.

Assuming the three files are in the current directory, the following script automates this process:

```
1 sudo cp itch-installer /usr/bin/
2
3 sudo mkdir -p /usr/lib/itch-installer/
4 sudo cp uninstalldb.py /usr/lib/itch-installer/
5 sudo cp updatedb.py /usr/lib/itch-installer/
6
7 sudo echo "installprefix="/usr"
8 tempdir="/tmp/itch/build"
9
10 pkgpath="/var/lib/itch/pkgs"
11 sharedir="/var/lib/itch/shared"
12
13 itchinstallerdir="/usr/lib/itch-installer"" > /etc/itch/itch.conf
```

5.3.2 The dependency resolver

The dependency resolver will come with three files: `itch-resolver`, `make-cnf.py`, and `get-positives.py`.

`itch-resolver` should be copied into `/usr/bin/`, and the two python scripts into `/usr/lib/itch-resolver/`. This directory should be made if not already present.

The configuration file should be created in `/etc/itch/itch-resolver.conf`, containing the fields specified in Section 5.1.

Assuming the three files are in the current directory, the following script automates this process:

```
1 sudo cp itch-resolver /usr/bin/
2
3 sudo mkdir -p /usr/lib/itch-resolver/
```

```

4 sudo cp make-cnf.py /usr/lib/itch-resolver/
5 sudo cp get-positives.py /usr/lib/itch-resolver/
6
7 sudo echo "installdir="/usr/lib/itch-resolver"
8
9 pkgmdir="/var/lib/itch-resolver"
10
11 itchdir="/var/lib/itch"
12
13 tempdir="/tmp/itch-resolver"" > /etc/itch/itch-resolver.conf

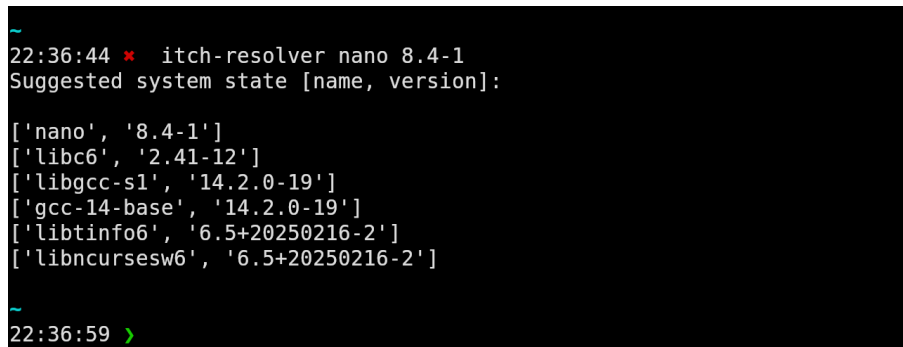
```

5.4 Example usage

To test the functionality of the package manager, I tested using all of its features with a large sample of packages, ensuring that it works generally and not just in a few particular cases. This section includes screenshots from some of these tests for one particular package, showcasing how *Itch* would work for a user wanting to install a package.

In this case, the package is a simple text editor called *nano*.

First, the dependency resolver is run with the package name and version, as in Figure 2.



```

~
22:36:44 ✖ itch-resolver nano 8.4-1
Suggested system state [name, version]:

['nano', '8.4-1']
['libc6', '2.41-12']
['libgcc-s1', '14.2.0-19']
['gcc-14-base', '14.2.0-19']
['libtinfo6', '6.5+20250216-2']
['libncursesw6', '6.5+20250216-2']
~
22:36:59 >

```

Figure 2: Output of the dependency resolver

This outputs a list of packages (and their versions) that should be installed for a working system state, having resolved the dependencies and any conflicts present, and taken into account the packages currently installed. This satisfies Aim 7. Since all of the packages listed in Figure 2 (bar *nano* itself) are all already installed, the user can move onto installing the software itself. If they were not, the user would have to install those packages first, using the same method they would use to install the target package. However, there is no need to resolve dependencies for those packages too, since that has already been incorporated in the system state output. This is the key benefit of the current system with a package database for a wide range of packages over a system where package dependency data is only given upon installation of that package. See Section 2.3 for more details.

To do this, they must download the source code of the package, compile it, and move to the directory containing the compiled source code. Only at that point do they invoke *itch-installer*. This allows them to compile the software themselves, and patch it as needed, meeting Aims 2 and 3. The output of the installer is shown in Figure 3, printing the output of the package's install method, prompting the user to confirm the package name, and to confirm the installation.

Figure 4 confirms that the *nano* package has been successfully installed and is working as desired.

At some point, the user may wish to uninstall the package. They can do this by issuing a command with

the `-u` flag to `itch-installer`. The output of this command is shown in Figure 5, listing the files that have been deleted in the process. After this, Figure 6 shows that the package is no longer available. This shows that Aim 6 has been met.

Finally, since `nano` has been installed one time, *Itch* will store it as a custom format package, allowing it to be reinstalled again without needing the original source code this time. This can be done by running `itch-installer` with the `-i` command and specifying the name of the package, as shown in Figure 7. This meets the requirements of Aim 4. It can again be verified that this works in Figure 8, again showing the program installed and working.


```

~/installs/nano-8.3 via C v15.2.1-gcc
22:28:43 > sudo itch-installer -i
Enter Password or Place finger on fingerprint reader:

Making install in doc
make[1]: Entering directory '/home/shayan/installs/nano-8.3/doc'
make install-recursive
make[2]: Entering directory '/home/shayan/installs/nano-8.3/doc'
make[3]: Entering directory '/home/shayan/installs/nano-8.3/doc'
make[4]: Entering directory '/home/shayan/installs/nano-8.3/doc'
make[4]: Nothing to be done for 'install-exec-am'.
/usr/bin/mkdir -p '/tmp/itch/build/usr/share/doc/nano-8.3'
/usr/bin/install -c -m 644 faq.html nano.1.html rnano.1.html nanorc.5.html nano.html '/tmp/itch/build/usr/share/doc/nano-8.3'
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/info'
/usr/bin/install -c -m 644 ./nano.info '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/info'
install-info --info-dir='/tmp/itch/build/home/shayan/prog/school/epq/dest/share/info' '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/info/nano.info'
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/man/man1'
/usr/bin/install -c -m 644 nano.1 rnano.1 '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/man/man1'
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/man/man5'
/usr/bin/install -c -m 644 nanorc.5 '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/man/man5'
make[4]: Leaving directory '/home/shayan/installs/nano-8.3/doc'
make[3]: Leaving directory '/home/shayan/installs/nano-8.3/doc'
make[2]: Leaving directory '/home/shayan/installs/nano-8.3/doc'
make[1]: Leaving directory '/home/shayan/installs/nano-8.3/doc'
Making install in lib
make[1]: Entering directory '/home/shayan/installs/nano-8.3/lib'
make install-recursive
make[2]: Entering directory '/home/shayan/installs/nano-8.3/lib'
make[3]: Entering directory '/home/shayan/installs/nano-8.3/lib'
make[4]: Entering directory '/home/shayan/installs/nano-8.3/lib'
make[4]: Nothing to be done for 'install-exec-am'.
make[4]: Leaving directory '/home/shayan/installs/nano-8.3/lib'
make[3]: Leaving directory '/home/shayan/installs/nano-8.3/lib'
make[2]: Leaving directory '/home/shayan/installs/nano-8.3/lib'
make[1]: Leaving directory '/home/shayan/installs/nano-8.3/lib'
Making install in m4
make[1]: Entering directory '/home/shayan/installs/nano-8.3/m4'
make[2]: Entering directory '/home/shayan/installs/nano-8.3/m4'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/home/shayan/installs/nano-8.3/m4'
make[1]: Leaving directory '/home/shayan/installs/nano-8.3/m4'
Making install in po
mv -f .deps/winio.Tpo .deps/winio.Po
gcc -g -O2 -Wall -o nano browser.o chars.o color.o cut.o files.o global.o help.o history.o move.o nano.o prompt.o rcfile.o search.o text.o
utils.o winio.o ../lib/libgnu.a -lz -lmagic -lncursesw
make[2]: Entering directory '/home/shayan/installs/nano-8.3/src'
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/bin'
/usr/bin/install -c nano '/tmp/itch/build/home/shayan/prog/school/epq/dest/bin'
make install-exec-hook
make[3]: Entering directory '/home/shayan/installs/nano-8.3/src'
cd /tmp/itch/build/home/shayan/prog/school/epq/dest/bin && rm -f rnano && ln -s nano rnano
make[3]: Leaving directory '/home/shayan/installs/nano-8.3/src'
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/home/shayan/installs/nano-8.3/src'
make[1]: Leaving directory '/home/shayan/installs/nano-8.3/src'
Making install in syntax
make[1]: Entering directory '/home/shayan/installs/nano-8.3/syntax'
make[2]: Entering directory '/home/shayan/installs/nano-8.3/syntax'
make[2]: Nothing to be done for 'install-exec-am'.
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/nano'
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/nano/extra'
/usr/bin/install -c -m 644 extra/ada.nanorc extra/fortran.nanorc extra/haskell.nanorc extra/povray.nanorc extra/spec.nanorc '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/nano/extra'
/usr/bin/mkdir -p '/tmp/itch/build/home/shayan/prog/school/epq/dest/share/nano'
/usr/bin/install -c -m 644 asm.nanorc autoconf.nanorc awk.nanorc changelog.nanorc cmake.nanorc c.nanorc css.nanorc default.nanorc elisp.nanorc
c email.nanorc go.nanorc groff.nanorc guile.nanorc html.nanorc java.nanorc javascript.nanorc json.nanorc lua.nanorc makefile.nanorc man.nanorc
markdown.nanorc nanohelp.nanorc nanorc.nanorc nftables.nanorc objc.nanorc ocaml.nanorc patch.nanorc perl.nanorc php.nanorc po.nanorc python.n
anorc ruby.nanorc rust.nanorc sh.nanorc sql.nanorc tcl.nanorc tex.nanorc texinfo.nanorc xml.nanorc yaml.nanorc '/tmp/itch/build/home/shayan/pr
og/school/epq/dest/share/nano'
make[2]: Leaving directory '/home/shayan/installs/nano-8.3/syntax'
make[1]: Leaving directory '/home/shayan/installs/nano-8.3/syntax'
make[1]: Entering directory '/home/shayan/installs/nano-8.3'
make[2]: Entering directory '/home/shayan/installs/nano-8.3'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/home/shayan/installs/nano-8.3'
make[1]: Leaving directory '/home/shayan/installs/nano-8.3'
Enter package name (blank for [nano-8.3]):

Proceed? (yes/NO): yes

~/installs/nano-8.3 via C v15.2.1-gcc
22:29:01 >

```

Figure 3: Output of the package installer

```
GNU nano 8.3                               New Buffer                               Modified
Hello, I am typing on the nano text editor.

All functions seem to be working, indicating it has been successfully installed.

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo      M-G Copy      ^B Where Was
```

Figure 4: A demonstration of the installed program

```
22:31:31 > itch-installer -u nano-8.3
./
rm: refusing to remove '.' or '..' directory: skipping '/home/shayan/prog/school/epq/dest./'
./bin/
./bin/rnano
./bin/nano
./share/
./share/nano/
./share/nano/yaml.nanorc
./share/nano/xml.nanorc
./share/nano/texinfo.nanorc
./share/nano/tex.nanorc
./share/nano/tcl.nanorc
./share/nano/sql.nanorc
./share/nano/sh.nanorc
./share/nano/rust.nanorc
./share/nano/ruby.nanorc
./share/nano/python.nanorc
./share/nano/po.nanorc
./share/nano/php.nanorc
./share/nano/perl.nanorc
./share/nano/patch.nanorc
./share/nano/ocaml.nanorc
./share/nano/objc.nanorc
./share/nano/nftables.nanorc
./share/nano/nanorc.nanorc
./share/nano/nanohelp.nanorc
./share/nano/markdown.nanorc
./share/nano/man.nanorc
./share/nano/makefile.nanorc
./share/nano/lua.nanorc
./share/nano/json.nanorc
./share/nano/javascript.nanorc
./share/nano/java.nanorc
./share/nano/html.nanorc
./share/nano/guile.nanorc
./share/nano/groff.nanorc
./share/nano/go.nanorc
./share/nano/email.nanorc
./share/nano/elisp.nanorc
./share/nano/default.nanorc
./share/nano/css.nanorc
```

Figure 5: Output of the uninstalling a package

```
~
22:32:51 > nano
bash: nano: command not found

~
22:32:52 * |
```

Figure 6: Uninstalled package no longer accessible

```
~
22:33:31 > sudo itch-installer -i nano-8.3
Proceed? (yes/NO): yes

~
22:33:38 >
```

Figure 7: Installing from a pre-existing package

```
GNU nano 8.3                               New Buffer                               Modified
Hello, I am typing on the nano text editor.
All functions seem to be working, indicating it has been successfully installed.

~C Help      ~O Write Out  ~F Where Is  ~K Cut       ~T Execute   ~G Location  ~U Undo      ~A Set Mark  ~I To Bracket
~X Exit      ~R Read File ~_ Replace   ~U Paste     ~J Justify   ~_ Go To Line ~E Redo      ~G Copy      ~G Where Was
```

Figure 8: A demonstration of the re-installed program

6 Conclusion

6.1 Extent of success

At the very start of the project, I established that a basic level of success can be established by the package manager being able to install packages. I have clearly succeeded in this, as shown in Section 5.4 - packages can be installed, uninstalled, and have their dependencies resolved.

However, this project can only be considered truly successful if it adds something on top of other, existing, package managers. In particular, my goal was to target it specifically for Linux From Scratch users. Based on that goal, I defined eight key non-technical aims, in Section 2.1, to guide the project, and serve as a metric to evaluate it upon completion.

Aims 2, 3, 4, and 6 were to be tested by using the package manager with a range of packages, and ensuring that they work in all cases. An example test is shown in Section 5.4, and tests on the packages *neovim*, *emacs*, *zsh*, *git*, *pulseaudio*, *perl*, and *minisat* were similarly successful. These are two text editors, a shell, a file-tracking utility, an audio driver, a programming language, and a SAT solver (which is actually the only dependency of *Itch* that is unavailable on a base LFS system). This test sample contained a diverse range of software, with different dependencies, conflicts, packages provided, and installation methods supported. Since *Itch* was successful in all of these cases, it is safe to consider these aims successful.

Aim 7, concerning the ability of the package manager to resolve dependencies, passed an initial check in the test cases above, where it successfully terminated and provided a list of packages that should be installed on a system with the target package. These system states were always successful in the above tests, allowing the target package to run correctly. I ensured that this was not accidental by also passing specific test cases with "hard" dependency situations (with many dependencies and conflicts between packages) using fake packages added to the test database to manually check whether a correct arrangement is output. These tests were all passed, satisfying me that this aim was also successful.

Aim 5 is the only aim that has not been fully successful. As detailed in Section 2.3, in order to create a usable dependency resolver, some form of dependency database is needed. In the Design section, I established this as one that will be held online, and must be downloaded once. Through the development stage, I realised that it is infeasible to create one wholly myself, since there are thousands of available packages, and currently available package managers, such as Pacman [2], have teams of developers working on maintaining their package databases. Therefore, users can solve this in three ways:

1. Creating their own package database,
2. Using another package manager's database,
3. Establishing a community *Itch* package database.

The first item is generally infeasible for most users, though many very dedicated Linux From Scratch will be able to do this, particularly if they only use a limited range of packages. The third solution is the ideal method - as I had initially aimed to do - however, it is beyond the scope of this project, and would take far too much time for me to do on my own. Therefore, for testing purposes, I used Debian's package database (Debian is a popular Linux distribution) modified to fit the format established in Section 4.1. In the long term, my hope is that I, and other users, can establish a large database to end this reliance.

While I do not believe that Aim 5 has been fully unsuccessful, since users can still make their own database of packages, the dependency resolver and package installer have been separated, and the program just needs the database to be downloaded one time (it does not itself interact with any online resource), it would be disingenuous to mark it as successful, as the most usable version of the dependency resolver relies on some form of externally held database.

Aim 1, the need to target Linux From Scratch systems and users, needs to be judged qualitatively, based on the development process I have undertaken, and how the finished product works. I believe that this has been very successful, as I have taken all possible steps to make *Itch* as suitable as possible for such systems and users. The following are just some of these decisions:

- The package manager does not compile any software for the user,
- The user can freely patch software,
- The user can add any software they desire to the package databases,
- The package manager does not interact with any online resources,
- The package database can be overridden - or even fully created - by the user,
- A fakeroot method is used for installation so that it is simple to understand by users and does not impose any methods of using the system,
- There is no curated repository of software versions that are supported - any software in any version that the user wants can be used,
- Updates are not mandated - the user can uninstall old versions and install new versions whenever they would like to,
- Precompiled binaries are not distributed or even supported,
- The only dependency used that is not provided in a basic Linux From Scratch installation is MiniSAT.

The final aim, Aim 8, is about there being a reason to use *Itch* specifically instead of any other existing package manager. In some ways, this defines whether the project as a whole has been successful, and something of value created. There is no one feature that has been implemented here that does not individually exist elsewhere, however, it is the specific combination that makes *Itch* unique. It is very specifically targeted to Linux From Scratch systems and users, as demonstrated above, and is a unique solution to the problems those users faced, as established in Section 1.2. Previously, they would need to either rely on another distribution's package manager - removing much of the control that Linux From Scratch affords a user - or install packages manually, if they did not want to create a basic package manager themselves. These are all impractical, particularly for very large packages such as desktop environments. *Itch* is in effect the solution to all of these problems. It handles the repetitive tasks of package removal and dependency resolution, that users find very difficult in larger systems, while leaving all other functions in the hands of the users, affording them as much control as possible. I am therefore happy to consider this aims, and the project as a whole, successful.

6.2 Scope for improvement

The following are examples of functionality that could be implemented in the future to improve the package manager.

- Speeding up the dependency resolver: a problem with the package manager right now is that the dependency resolver runs slowly - taking seconds for a task that should ideally take milliseconds. This could be improved upon in two ways. Firstly, I could spend more time tweaking the resolution algorithm, to try and find optimisations that can speed up the process. Secondly, rewriting the resolver in a compiled language such as C or C++ would significantly improve the speed compared to Python that I am using at the moment.

- Allowing currently installed packages to be marked as such: while ideal, it is unlikely that users will install this package manager before they have already installed a whole array of other packages manually. Currently, they will have to manually modify the database of installed packages so that *Itch* knows these are installed, and avoids conflicts with them when resolving dependencies for new packages. Although this is likely within the capacity of most Linux From Scratch users, it would be nice to add in functionality to add these in natively, without the user having to deal with the databases themselves.
- Introducing functionality to natively allow updating packages could help to make *Itch* easier to use. Updating is currently possible, but it takes two separate commands: one to uninstall the previous version, and another to install the new version. Implementing a single command to handle both could save users some time.
- A method to "export" a file containing details of all packages installed on one system, and another method to install all the packages from that file, would allow users to distribute their created system between devices. On one hand, this could be seen as putting too much functionality into one program, which is against the Unix philosophy [15], but on the other hand, this is a natural extension of a package manager, since packages installed in effect defines a system's state.
- Including a guided installer that simplifies the installation process for *Itch* could help to eliminate any errors caused by improper installation, however it is very low priority since this should not pose any sort of challenge to a user that has managed to install a whole Linux From Scratch system.

6.3 Comparison with other package managers

Here, I will compare *Itch* with two very popular package managers: *Pacman* [2], for the Arch Linux distribution, and *Apt* [26], for Debian.

Category	Apt	Pacman	Itch
Are precompiled packages distributed?	Yes	Yes	No. The user downloads and compiles source code, and the package manager just takes care of installation. Packages, however, are made from code that the user compiles and installs, for subsequent re-installation.
Installation procedure	Extracts the precompiled archive in place	Extracts the precompiled archive in place	Installs packages in a fake directory first, creating an archive of the files to install, before copying them into place
Uninstallation procedure	Uses the package archive to remove all files specified	Uses the package archive to remove all files specified	Uses the package archive created during installation to remove all files specified
Allows patching source code of packages	No	No	Yes
Allows users to compile their own packages	No	No	Yes

Package information database	Holds a centrally maintained database	Holds a centrally maintained database	Either a centrally maintained database (to be created) or a user's own database can be used
Packages available	Packages in a specific version defined by the version of Debian installed. These are typically older, to maintain stability.	The newest possible version of the package that has been packaged for Arch Linux, providing the newest software at the expense of stability.	Whichever packages the user can find dependency installation for. They can choose older, more stable versions, or brand-new, development versions with no restriction.
Updates	Most updates are undertaken as full-system upgrades, with a new Debian version allowing a newer version of all packages.	Updates are undertaken on a rolling basis, forced whenever new packages are being installed, ensuring the newest software is always installed.	Software can be updated, or not updated, as and when the user wishes. There are no specific requirements, giving the user full control.
Overall structure	The underlying package installation and dependency resolution is performed by <i>dpkg</i> , while <i>Apt</i> handles the other maintenance and frontend tasks.	The package management is all handled together by <i>Pacman</i> .	<i>itch-installer</i> handles the package installation and removal, while <i>itch-resolver</i> resolves dependencies. This is a unique method of separation, allowing users to choose which functionality they feel they would benefit from.

Overall, *Itch* has been designed to leave as much as possible in the hands of the user. The structures of *Apt* and *Pacman* define the Debian and Arch Linux operating systems respectively, by imposing a usage structure upon users, whether it be one of utmost stability or modernity. However, *Itch* intentionally avoids these decisions, leaving it to the user to create their own distribution, as the Linux From Scratch project intended. It is unique in allowing users to maintain full control of their systems, while automating some of the most tiresome tasks that previously made the maintenance of a Linux From Scratch system almost infeasible over the long term.

A Package installer

A.1 itch-installer

```
1  #!/bin/bash
2
3  source "/etc/itch/itch.conf"
4
5  filedbdir="$sharedir/files.gdbm"
6  pkgdbdir="$sharedir/pkgs.gdbm"
7
8  pythonupdatedir="$itchinstallerdir/updatedb.py"
9  pythonuninstalldir="$itchinstallerdir/uninstalldb.py"
10
11 configure() {
12     echo "configure as normal, ensuring the following is added:"
13     echo "    --prefix=$installprefix"
14 }
15
16 build() {
17     echo "make as normal"
18 }
19
20 install() {
21     rm -rf "$tempdir" # in case not properly cleaned before
22     mkdir -p $tempdir
23
24     make DESTDIR="$tempdir" install # install in temp dir
25
26     findname
27
28     sudo mkdir -p "$pkgpath/$pkgname"
29
30     sudo tar czf "$itchpkgpath" -C "$tempdir$installprefix" .
31
32     installpkg
33 }
34
35 installpkg() { # existing tarball in pkgpath
36     cd "$installprefix"
37
38     pkgfiles=$(tar tzf "$itchpkgpath")
39
40     # finds existing files that will be overwritten - credit
41     # https://github.com/skeeto/dotfiles/blob/master/bin/qpkg (with significant modifications)
42     for file in $(tar tzf "$itchpkgpath" | grep -v '/$'); do
43         if [ -e "$file" ]; then
44             printf "Overwrites '%s'\n" "${file#./}"
45         fi
46     done
47
48     read -p "Proceed? (yes/NO): " confirmation
49
50     if [ "${confirmation,,}" = "yes" ] || [ "${confirmation,,}" = "y" ]; then # ${[string],,}
51         # converts to lower case
52     fi
53 }
```



```

50     updateshared
51     tar xf "$itchpkgpath" # files extracted into install dir
52 else
53     echo "Aborted!"
54 fi
55
56 rm -rf "$tempdir"
57 }
58
59 uninstall() {
60     pkgname_name="${pkgname%%-[0-9]*}"
61
62     gdbmout="$(echo "fetch $pkgname_name" | sudo gdbmtool "$pkgdbdir" 2>/dev/null)" # empty if
63     not existing
64
65     if [ "$gdbmout" = "" ]; then # CHECK THIS LINE, NEW!
66         echo "Error! Package not installed to begin with!"
67         exit
68     fi
69
70     pkgfiles="$(tar tzf "$itchpkgpath")"
71
72     toremove="$(printf "%s" "${pkgfiles[@]}" | sudo python "$pythonuninstalldir" "$filedbdir"
73     "$pkgname" "$itchpkgpath")"
74
75     IFS=$'\n' read -rd '' -a tomovearr <<< "$toremove"
76
77     for file in "${tomovearr[@]}; do
78         echo "$file"
79         sudo rm -rf "$installprefix/$file"
80     done
81
82     echo "delete $pkgname_name" | sudo gdbmtool "$pkgdbdir" 2>/dev/null
83 }
84
85 findname() {
86     pkgname=$(basename "$(pwd)")
87
88     # tries to find package name (either current dir or parent if in build dir)
89     if [ "$pkgname" = "build" ]; then
90         pkgname=$(basename "$(dirname "$(pwd)")")
91     fi
92
93     echo "Enter package name (blank for [$pkgname]):"
94     read input
95
96     if [ "$input" != "" ]; then
97         pkgname="$input"
98     fi
99
100     itchpkgpath="$pkgpath/$pkgname/$pkgname.itchpkg"
101 }
102
103 getnamepkgd() { # get name for a pkg that has been packaged into a .itchpkg
104     pkgname="$2"
105 }

```

```

104 itchpkgpath="$pkgpath/$pkgname/$pkgname.itchpkg"
105
106 if [ ! -f "$itchpkgpath" ]; then
107     echo "Error! Package archive not found!"
108     exit
109 fi
110 }
111
112 updateshared() {
113     # create dir if not already existing | python handles creation of non-existent db
114     sudo mkdir -p "$sharedir"
115
116     # root due to location of file made
117     printf "%s" "${pkgfiles[@]}" | sudo python "$pythonupdatedir" "$filedbdir" "$pkgname"
118
119     pkgname_name="${pkgname%-[0-9]*}"
120     pkgname_version="${pkgname#${pkgname_name}-}"
121
122     # define the pkg as installed with version given
123     echo "store $pkgname_name $pkgname_version" | sudo gdbmtool "$pkgdbdir" 2>/dev/null
124 }
125
126 case "$1" in
127     "-c")
128         configure ;;
129     "-b")
130         build ;;
131     "-i")
132         if [ -z "$2" ]; then
133             install
134         else
135             getnamepkgd "$@"
136             installpkg
137         fi ;;
138     "-u")
139         if [ -z "$2" ]; then
140             echo "Error!"
141         else
142             getnamepkgd "$@"
143             uninstall "$@"
144         fi ;;
145     *)
146         echo "Error!"
147         ;;
148 esac

```

A.2 updatedb.py

```

1 import sys
2 import dbm.gnu
3
4 dbdir = sys.argv[1]
5 pkgname = sys.argv[2]
6

```

```

7 pkgfiles = [line.strip() for line in sys.stdin]
8
9 db = dbm.gnu.open(dbdir, 'c');
10
11 for file in pkgfiles:
12     gdbmout = db.get(file)
13
14     if gdbmout is not None:
15         pkgs = gdbmout.decode().split(' ')
16     else:
17         pkgs = []
18
19     # don't add pkgname if already existing - eg for reinstallations
20     if pkgname not in pkgs:
21         pkgs.append(pkgname)
22
23     db[file] = " ".join(pkgs)
24
25 db.close()

```

A.3 uninstalldb.py

```

1 import sys
2 import dbm.gnu
3
4 dbdir = sys.argv[1]
5 pkgname = sys.argv[2]
6 itchpkgpath = sys.argv[3]
7
8 pkgfiles = [line.strip() for line in sys.stdin]
9
10 db = dbm.gnu.open(dbdir, 'c');
11
12 for file in pkgfiles:
13     gdbmout = db.get(file)
14
15     if gdbmout is not None:
16         pkgs = gdbmout.decode().split(' ')
17     else: # must have been an error
18         pkgs = []
19         continue
20
21     pkgs.remove(pkgname)
22
23     if len(pkgs) == 0:
24         print(file)
25         del db[file]
26     else:
27         db[file] = " ".join(pkgs)
28
29 db.close()

```

B Dependency resolver

B.1 itch-resolver

```
1  #!/bin/bash
2
3  source "/etc/itch/itch-resolver.conf"
4
5  mkdir -p "$tempdir"
6
7  sudo python "$installdir/make-cnf.py" "$pkgdir/" "$itchdir/" "$1" "$2"
8      "$tempdir/mappings.json" > "$tempdir/output" # inputs are pkg and version
9  minisat "$tempdir/output" "$tempdir/minisat_out" > /dev/null # don't print sat solver output
10 (grep -oE '(^| ) [0-9]+' "$tempdir/minisat_out" | tr -d ' ') > "$tempdir/positives"
11
12 python "$installdir/get-positives.py" "$tempdir/mappings.json" "$tempdir/positives"
13
14 rm -rf "$tempdir"
```

B.2 make-cnf.py

```
1  import sys
2  import dbm.gnu
3  import re
4  import os
5  from pathlib import Path
6  import json
7
8  core_dir = sys.argv[1] # /var/lib/itch-resolver/
9  global_db_dir = core_dir + "global.gdbm"
10 pkg_dir = core_dir + "pkgs/"
11
12 local_pkg_dir = sys.argv[2] # /var/lib/itch/
13 local_pkg_db_dir = local_pkg_dir + "shared/pkgs.gdbm" # /var/lib/itch/shared/pkgs.gdbm
14 local_pkg_dir = local_pkg_dir + "pkgs/"
15
16 new_pkg_name = sys.argv[3]
17 new_pkg_version = sys.argv[4] # NOTE: new pkg MUST be in global db
18
19 global_db = dbm.gnu.open(global_db_dir, 'c')
20
21 sat_clauses = list()
22
23 # mapping of pkgs to CNF number
24 pkg_to_num = dict()
25 num_to_pkg = dict()
26
27 pkg_to_deps = dict()
28 pkg_to_versions = dict()
29 pkg_to_deps = dict()
30 pkg_to_conflicts = dict()
31 pkg_to_provides = dict()
32
```

```

33 def build_pkg_dicts():
34     return
35
36 def get_versions(pkg):
37     dir = pkgs_dir + pkg
38
39     if pkg in pkg_to_versions:
40         return pkg_to_versions[pkg]
41
42     if not Path(dir).exists():
43         return []
44
45     versions = os.listdir(dir)
46
47     pkg_to_versions[pkg] = versions
48
49     return versions
50
51 def version_in_range(version, lower, upper):
52     if compare_versions(version, lower) == "<":
53         return False
54     if upper and compare_versions(version, upper) == ">":
55         return False
56
57     return True
58
59 def compare_versions(version_1, version_2):
60     if version_1 == version_2:
61         return "="
62
63     v1a, v1b, v1c = split_version(version_1)
64     v2a, v2b, v2c = split_version(version_2)
65
66     if v1a > v2a:
67         return ">"
68     if v1a < v2a:
69         return "<"
70
71     for i in range(0, min(len(v1b), len(v2b))):
72         if v1b[i] > v2b[i]:
73             return ">"
74         if v1b[i] < v2b[i]:
75             return "<"
76
77     if len(v1b) > len(v2b):
78         return ">"
79     if len(v1b) < len(v2b):
80         return "<"
81
82     if v1c > v2c:
83         return ">"
84     if v1c < v2c:
85         return "<"
86
87     return "="
88

```

```

89 def split_version(version):
90     if ":" in version:
91         epoch_str, version = version.split(":", 1)
92         epoch = int(epoch_str)
93     else:
94         epoch = 0
95
96     if "+" in version:
97         version = version.split("+", 1)[0]
98
99     if "-" in version:
100         main_str, end_str = version.rsplit("-", 1) # rsplit gets last occurrence
101     else:
102         main_str, end_str = version, "0"
103
104     # split into array by .
105     main = [int(x) for x in main_str.split(".") if x.isnumeric()]
106
107     # strip text at end of c - only present in debian files
108     m = re.match(r"(\d+)", end_str)
109     end = int(m.group(1)) if m else 0
110
111     return epoch, main, end
112
113
114 # encode "dependencies must be met, or the package not be installed"
115 def encode_deps(pkg, version):
116     dep_to_versions = get_deps(pkg, version)
117
118     for dep, vers in dep_to_versions.items():
119         new_clause = "-" + pkg_to_num[pkg, version] + " "
120
121         entered = False
122
123         for ver in vers:
124             entered = True
125             new_clause += pkg_to_num[dep, ver] + " "
126
127         new_clause += "0"
128
129         if (entered):
130             sat_clauses.append(new_clause)
131
132 # encode "conflicts must not exist, or the package not be installed"
133 def encode_conflicts(pkg, version):
134     conflict_to_versions = get_conflicts(pkg, version)
135
136     for dep, vers in conflict_to_versions.items():
137         new_clause = "-" + pkg_to_num[pkg, version] + " "
138
139         entered = False
140
141         for ver in vers:
142             entered = True
143
144             if (dep, ver) in provides:

```

```

145         for providing_pkg, providing_ver in provides[dep, ver]:
146             new_clause += "-" + pkg_to_num[providing_pkg, providing_ver] + " "
147         else:
148             new_clause += "-" + pkg_to_num[dep, ver] + " " # not pkg or not conflict
149
150     new_clause += "0"
151
152     if (entered):
153         sat_clauses.append(new_clause)
154
155 def get_deps(pkg, version):
156     pkg_dir = pkgs_dir + pkg + "/" + version + "/"
157     dep_file = pkg_dir + "depends.itch"
158
159     if (pkg, version) in pkg_to_deps:
160         return pkg_to_deps[pkg, version]
161
162     results = dict()
163
164     if not Path(dep_file).exists():
165         pkg_to_deps[pkg, version] = results
166         return results
167
168     with open(dep_file) as file:
169         for line in file:
170             fields = line.strip().split()
171
172             dep_name = fields[0]
173             lower = fields[1]
174             upper = ""
175
176             if len(fields) > 2:
177                 upper = fields[2]
178
179             dep_versions = []
180
181             for ver in get_versions(dep_name):
182                 if version_in_range(ver, lower, upper):
183                     dep_versions.append(ver)
184
185             results[dep_name] = dep_versions # maps dependency name -> list of allowed versions
186
187     pkg_to_deps[pkg, version] = results
188
189     return results
190
191 def get_conflicts(pkg, version):
192     pkg_dir = pkgs_dir + pkg + "/" + version + "/"
193     conflict_file = pkg_dir + "conflicts.itch"
194
195     results = dict()
196
197     if not Path(conflict_file).exists():
198         return results
199
200     with open(conflict_file) as file:

```

```

201     for line in file:
202         fields = line.strip().split()
203
204         conflict_name = fields[0]
205         lower = fields[1]
206         upper = ""
207
208         if len(fields) > 2:
209             upper = fields[2]
210
211         conflict_versions = []
212
213         for ver in get_versions(conflict_name):
214             if version_in_range(ver, lower, upper):
215                 conflict_versions.append(ver)
216
217         results[conflict_name] = conflict_versions # maps conflict name -> list of not
allowed versions
218
219     return results
220
221 def set_provides(pkg, version):
222     pkg_dir = pkgs_dir + pkg + "/" + version + "/"
223     provide_file = pkg_dir + "provides.itch"
224
225     if not Path(provide_file).exists():
226         return
227
228     with open(provide_file) as file:
229         for line in file:
230             provided_pkg, provided_ver = line.strip().split()
231
232             if (provided_pkg, provided_ver) in provides:
233                 provides[provided_pkg, provided_ver].append((pkg, version))
234             else:
235                 provides[provided_pkg, provided_ver] = [(pkg, version)]
236
237 def get_closure(pkg, ver):
238     start = (pkg, ver)
239     queue = [start]
240     visited = set()
241
242     while len(queue) > 0:
243         (p, v) = queue.pop()
244
245         if (p, v) in visited:
246             continue
247
248         visited.add((p, v))
249
250         deps = get_deps(p, v)
251         for dep in deps.keys():
252             for allowed_ver in deps[dep]:
253                 queue.append((dep, allowed_ver))
254
255     return visited

```



```

256
257
258 pkgs = global_db.keys()
259 provides = dict() # mapping of (provided pkg, provided pkg version) -> list of (pkg, version)
260
261 count = 1
262 for pkg in pkgs:
263     versions = global_db[pkg.decode()].decode().split()
264
265     for version in versions:
266         pkg_to_num[pkg.decode(), version] = str(count)
267         num_to_pkg[count] = (pkg.decode(), version)
268
269         count += 1
270
271         set_provides(pkg.decode(), version)
272
273 for pkg in pkgs: # needs second loop so mappings (and provides) are complete!
274     versions = global_db[pkg.decode()].decode().split()
275
276     for version in versions:
277         encode_deps(pkg.decode(), version)
278         encode_conflicts(pkg.decode(), version)
279
280     # encode "only one version of each package"
281     for i in range(0, len(versions)):
282         for j in range(i+1, len(versions)):
283             sat_clauses.append("-" + pkg_to_num[pkg.decode(), versions[i]] + " "
284                               + "-" + pkg_to_num[pkg.decode(), versions[j]] + " 0")
285
286 local_pkgs_db = dbm.gnu.open(local_pkgs_db_dir, 'c')
287
288 local_pkgs = local_pkgs_db.keys()
289
290 full_closure = set()
291
292 for pkg in local_pkgs:
293     # encode "each package installed should be installed (but whatever version)"
294     versions = global_db[pkg.decode()].decode().split() # all available versions
295
296     new_clause = ""
297     for version in versions:
298         new_clause += pkg_to_num[pkg.decode(), version] + " "
299
300         full_closure.update(get_closure(pkg.decode(), version)) # get closure of each
301         # installed/installable pkg-ver
302         new_clause += "0"
303
304     sat_clauses.append(new_clause)
305
306 full_closure.update(get_closure(new_pkg_name, new_pkg_version))
307
308 # encode "pkg to install should be installed with GIVEN version"
309 sat_clauses.append(pkg_to_num[new_pkg_name, new_pkg_version] + " 0")
310
311 for pkg_num, (pkg, version) in num_to_pkg.items():

```

```

311     if (pkg, version) not in full_closure:
312         sat_clauses.append("-" + str(pkg_num) + " 0")
313
314 for clause in sat_clauses:
315     print(clause)
316
317 num_to_pkg_closure = dict()
318
319 for (pkg, ver) in full_closure:
320     num_to_pkg_closure[pkg_to_num[pkg, ver]] = (pkg, ver)
321
322 with open(sys.argv[5], 'w') as file:
323     json.dump(num_to_pkg_closure, file)

```

B.3 get-positives.py

```

1  import json
2  import sys
3
4  print("Suggested system state [name, version]:\n")
5
6  with open(sys.argv[1], 'r') as file:
7      num_to_pkg = json.load(file)
8
9  with open(sys.argv[2], 'r') as file:
10     for line in file:
11         line = line.rstrip() # remove trailing \n
12
13         if line == "0":
14             break
15
16     print(num_to_pkg[line])

```

References

- [1] Richard Stallman. *Linux and the GNU System*. 2nd Nov. 2021. URL: <https://www.gnu.org/gnu/linux-and-gnu.en.html> (visited on 26/08/2025).
- [2] Arch Linux contributors. *pacman*. 6th Sept. 2025. URL: <https://wiki.archlinux.org/title/Pacman> (visited on 07/09/2025).
- [3] Linux From Scratch. *Welcome to Linux From Scratch!* 2025. URL: <https://www.linuxfromscratch.org/> (visited on 24/08/2025).
- [4] Linux From Scratch. *Package Management*. 5th Mar. 2025. URL: <https://www.linuxfromscratch.org/lfs/view/stable/chapter08/pkgmgt.html> (visited on 24/08/2025).
- [5] Ben Van Daele. *Pacman on LFS 8.1*. 1st Aug. 2019. URL: <https://github.com/benvd/lfs-pacman?tab=readme-ov-file> (visited on 24/08/2025).
- [6] Reddit. *I'm curious as to how many linux users can code*. 2022. URL: <https://archive.is/2tcxS> (visited on 07/09/2025).
- [7] Tushar Teredesai. *Fakeroot approach for package installation*. 17th Jan. 2006. URL: <https://www.linuxfromscratch.org/hints/downloads/files/fakeroot.txt> (visited on 20/08/2025).
- [8] Chris Wellons. *A Crude Personal Package Manager*. 27th Mar. 2018. URL: <https://nullprogram.com/blog/2018/03/27/> (visited on 20/05/2025).
- [9] Pierre Hebert. *TRIP, a TRivial Packager for LFS (and other linux systems)*. 13th May 2006. URL: https://www.linuxfromscratch.org/hints/downloads/files/package_management_using_trip.txt (visited on 20/08/2025).
- [10] NixOS contributors. *Why You Should Give it a Try*. URL: <https://nixos.org/guides/nix-pills/01-why-you-should-give-it-a-try.html> (visited on 18/08/2025).
- [11] Pietro Abate et al. *Dependency Solving: a Separate Concern in Component Evolution Management*. Univ Paris Diderot and INRIA Paris-Rocquencourt, 2012.
- [12] Arch Linux contributors. *Arch Linux*. 3rd Sept. 2025. URL: https://wiki.archlinux.org/title/Arch_Linux (visited on 20/09/2025).
- [13] Debian contributors. *WhyDebian*. 3rd July 2022. URL: https://wiki.debian.org/WhyDebian#Utility_and_usability (visited on 20/09/2025).
- [14] Gentoo contributors. *ebuild repository*. 29th May 2025. URL: https://wiki.gentoo.org/wiki/Ebuild_repository#The_Gentoo_ebuild_repository (visited on 21/09/2025).
- [15] Mike Gancarz. "I. The Unix Philosophy". In: Dec. 2003. ISBN: 9781555582739. DOI: 10.1016/B978-155558273-9/50003-8.
- [16] The Linux Foundation. *Filesystem Hierarchy Standard*. 19th Mar. 2015. URL: https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.pdf (visited on 10/09/2025).
- [17] GNU. *GNU Tar*. 18th July 2023. URL: <https://www.gnu.org/software/tar/> (visited on 10/09/2025).
- [18] Chris Wellons. *qpkg*. 27th Jan. 2023. URL: <https://github.com/skeeto/dotfiles/blob/master/bin/qpkg> (visited on 20/05/2025).
- [19] Pierre Gaumond et al. *GNU dbm*. 2025. URL: <https://www.gnu.org.ua/software/gdbm/manual/gdbm.pdf> (visited on 03/09/2025).
- [20] Linux From Scratch. *GDBM-1.26*. 1st Sept. 2025. URL: <https://www.linuxfromscratch.org/lfs/view/stable-systemd/chapter08/gdbm.html> (visited on 03/09/2025).
- [21] Python Software Foundation. *dbm — Interfaces to Unix "databases"*. 11th Sept. 2025. URL: <https://docs.python.org/3/library/dbm.html#module-dbm.gnu> (visited on 11/09/2025).

- [22] Niklas Eén and Niklas Sörensson. “An Extensible SAT-solver”. In: *Theory and Applications of Satisfiability Testing*. Ed. by Enrico Giunchiglia and Armando Tacchella. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 502–518.
- [23] David A. Wheeler. *MiniSAT User Guide*. 28th June 2008. URL: <https://dwheeler.com/essays/minisat-user-guide.html> (visited on 23/09/2025).
- [24] Robert Atkey. *Package Installation Problem*. URL: <https://personal.cis.strath.ac.uk/robert.atkey/cs208/packages.html> (visited on 23/09/2025).
- [25] John Burkardt. *CNF Files*. URL: <https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html> (visited on 02/10/2025).
- [26] Debian Contributors. *The APT package management tool*. URL: <https://salsa.debian.org/apt-team/apt> (visited on 02/01/2026).